

third-party caveats, an inner, nested discharge macaroon need not be included in the discharge for the outer caveat. Thus, for example, if a third party caveat may restrict access to users within a certain age range, the caveat discharging service may add a caveat that requires a third-party authority to assert the bearer’s date of birth; subsequently, the same discharging service may remove this birthdate-assertion caveat when it issues a discharge macaroon, to not reveal the bearer’s age, thereby protecting their privacy.

With regards to privacy, it should be noted that attackers may try to add third-party caveats to probe for private information—e.g., by tricking users to reveal membership in some group. However, for this, attackers must be able to both trigger users to make requests with certain parameters, and observe the results. Under these conditions, similar attacks are equally possible in many existing authorization frameworks (e.g., SPKI/SDSI), and not specific to macaroons.

D. Local Discharging of Third-party Caveats

Third-party caveats can be used to implement decentralized authorization using holder-of-key proofs from authentication servers, as explained in earlier examples. However, third-party caveats may be discharged not just by networked servers, but by any isolated protection domain capable of holding secrets—such as, for example, a Web browser frame or extension, a mobile application, or a hardware token that is available locally to a user at a client.

The delegation of third-party caveat discharging to such local principals can improve both performance (by reducing network messages), as well as the precision of authorization policy enforcement. For example, a collection of unrelated Web services may use the same federated login system; if each service maintains separate sessions, logging out of one service may leave the user, unknowingly, still logged in and authorized in other services, and thereby put them at risk. Instead, using macaroons, each request, on all sessions, may derive a fresh, or very short-lived, caveat discharge from the third-party federated login service (e.g., via a hidden `iframe` that holds the caveat root key to derive discharge macaroons and is accessible from each service’s Web page). Thus, a federated login service may hold the authoritative credentials, and services may be decoupled, yet the user need log out only once to immediately de-authorize all services.

E. Revocation and Versioning

As previously mentioned in Section II, macaroons excel at the common revocation strategies for credential-based authorization: using short-lived, fresh credentials, that can be subject to state-based checking and can be split between principals [48, Chapter 9.8]. Indeed, the local discharging of caveats of the previous subsection is just one example of split credentials, subject to state-based checks. For other workloads or desired policies, revocation may be done via a database “whitelist” (resp. “blacklist”) of valid macaroons (resp. invalid), via embedding expiry times or epoch counters in macaroons, or via some combination of the above.

Commonly, state-based revocation may be via a third-party caveat to a service like a network time authority. To avoid the need for redundant discharging, such caveats might—when

they encode a monotonic property—use the same caveat identifier (and, hence, the same root key), for multiple macaroons, without risk. Thus, for example, a Web browser might hold hundreds of macaroons for images at the same Cloud service, such that all macaroons share the same third-party revocation-check caveat, and a single macaroon can discharge that caveat for any image. Of course, this discharge macaroon can, and should, be very short lived.

Revocation may also be required when target services are updated. For example, the operation `delete` may be added to a target service that offers previously-immutable, read-only storage, and the existing macaroons for this target may not constrain the operations in requests. In this case, the service may implement a default-deny policy for the `delete` operation, and authorizing its use only in future macaroons. As a more principled approach, targets may *version* aspects of their service, and embed first-party predicates in macaroons that constrain the authorized versions.

F. Disjunctive and Negated Caveats

When adding third-party caveats to a macaroon, it may be desirable to add a set of them where only one needs to be discharged. An example use case would be to allow a client to choose which authentication provider to use. One way of achieving this would be to augment macaroons to also include a conjunction or disjunction operator at the beginning of every caveat list. The `Verify` method would then accordingly conjunct or disjunct the proof obligations specified by the caveats. This leads to a notion of caveats on a macaroon as a logical formula with both disjunction and conjunction.

One may ask if it makes sense to add negation as well, where a macaroon can only be invoked if a certain caveat is *not* discharged. However, the semantics of negation become tricky when the negated caveat discharge macaroon contains further nested third-party caveats. Thus, negation should only be supported at the level of predicate checking, e.g., along the lines of `op ∉ {write, delete}`, if needed.

G. Caveats on the Structure of Macaroons

In some applications, there may be a need for caveat predicates to make assertions about the structure and values in the set of authorizing macaroons. In this case, one macaroon’s caveats could restrict the shape of the tree of macaroons that is induced during verification at the target service, or the values permitted in another macaroon in that tree.

Such a *structural caveat* may, for example, be used to enforce notions of well-formed transactions, by ensuring that a certain order of nested third-party caveat discharges are present [15]. Structural caveats may also name values in other caveats or discharge macaroons. For example, the BrowserID protocol behind Mozilla Persona [42] states that identity assertions (minted by a Web browser and verified by a trusted service) must successfully verify only if they state the domain name of the relying party they are meant for. Recasting BrowserID in terms of macaroons, this check can be enforced in the form of a structural caveat.

Structural caveats can also be used to make assertions about public-key signatures. For example, a service may mint